

SQL

Carina F. Dorneles
dorneles@inf.ufsc.br

Exemplo

► Modelo Lógico – relacional

```
cidade (codigo, nome, UF)
medico (codigo, nome, email, CRM, codCid#)
    codCid REFERENCIA cidade (codigo)
paciente (codigo, nome, email, fone, codCid#)
    codCid REFERENCIA cidade (codigo)
consulta (data, hora, codPac#, codMed#)
    codPac REFERENCIA paciente (codigo)
    codMed REFERENCIA medico (codigo)
medicamento (codigo, descricao)
cons_medicame (data, hora, codPac#, codMedica#)
    codMedica REFERENCIA medicamento (codigo)
    (data, hora, codPac) REFERENCIA consulta (data, hora, codPac))
```



Manipulação das tabelas

- ▶ CREATE TABLE

- ▶ Cria tabelas no Banco de Dados

- ▶ ALTER TABLE

- ▶ Altera tabelas já existentes no Banco de Dados

- ▶ DROP TABLE

- ▶ Apaga tabelas já existentes no Banco de Dados



Criação de tabelas

- ▶ Exemplo

cidade (codigo, nome, UF)

- ▶ Em SQL, fica:

```
CREATE TABLE cidade
(
    codigo integer not null,
    nome varchar (40),
    UF char(2)
);
```



Criação de tabelas

Nome do atributo Tipo do atributo

Restrições, que podem ser:
NOT NULL, DEFAULT, CHECK(), etc...

```
CREATE TABLE cidade
```

```
(
```

```
    [codigo] [integer] [not null],
```

```
    nome varchar (40) ,
```

```
    UF char (2) ,
```

```
    PRIMARY KEY (codigo)
```

```
);
```

Usar quando há outro comando SQL abaixo



Tipos

- ▶ Os tipos, normalmente, são dependentes do SGBD. Alguns são padrão SQL.
- ▶ **Numéricos Inteiros**
 - ▶ `int` (ou `integer`)
 - ▶ `longint`
 - ▶ `smallint`
- ▶ **Numéricos com casas decimais**
 - ▶ `numeric`
 - ▶ `decimal`
- ▶ **Caracteres**
 - ▶ `char(total)` - string de tamanho exato total
 - ▶ `varchar(total)` - string de tamanho máximo total
- ▶ **Data e hora**
 - ▶ `date`
 - ▶ `time`
 - ▶ `timestamp`



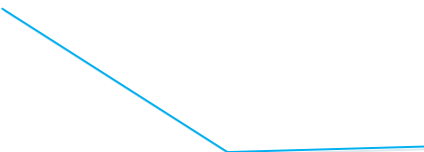
Criação de tabelas

- ▶ Exemplo

cidade (codigo, nome, UF)

- ▶ Em SQL, fica:

```
CREATE TABLE cidade  
(  
    codigo integer not null,  
    nome varchar (40),  
    UF char(2)  
);
```



**Obs.: a tabela ainda não tem
chave primária**

Chave primária (PK)

► Definição da chave primária (PK)

1. definir os atributos

2. daqueles atributos, qual faz parte da chave primária (PK)

► Exemplo: a seguinte definição:

cidade (codigo, nome, UF)

► Em SQL, fica:

```
CREATE TABLE cidade
```

```
(
```

```
    codigo integer not null,
```

```
    nome varchar (40),
```

```
    UF char(2),
```

```
    PRIMARY KEY(codigo)
```

```
);
```

Definir todos os atributos

Dos atributos definidos, definir qual(is) faz(em) parte da PK

Alternativa para definição de PK

```
CREATE TABLE cidade  
(  
    codigo integer not null PRIMARY KEY,  
    nome varchar (40),  
    UF char(2)  
);
```

Obs.:

- Primeiro o atributo é definido,
depois é feita a definição de PK



Tabelas com FK

▶ Com **chave estrangeira** (FK)

▶ Exemplo: a seguinte definição:

```
medico (codigo, nome, email, CRM, codCid#)  
      codCid REFERENCIA cidade (codigo)
```



Tabelas com FK

▶ Com **chave estrangeira** (FK)

▶ Exemplo: a seguinte definição:

```
medico (codigo, nome, email, CRM, codCid#)  
      codCid REFERENCIA cidade (codigo)
```

▶ Em SQL, fica:

```
CREATE TABLE medico  
(  
    codigo integer not null,  
    nome varchar (40),  
    email varchar(20),  
    CRM integer,  
    codcid integer,  
    PRIMARY KEY(codigo),  
    FOREIGN KEY(codcid) REFERENCES cidade (codigo)  
)
```



Tabelas com FK

► Com **chave estrangeira** (FK)

► Exemplo: a seguinte definição:

```
medico (codigo, nome, email, CRM, codCid#)  
      codCid REFERENCIA cidade (codigo)
```

► Em SQL, fica:

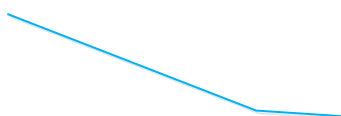
```
CREATE TABLE medico  
(  
    codigo integer not null,  
    nome varchar (40),  
    email varchar(20),  
    CRM integer,  
    codcid integer,  
    PRIMARY KEY(codigo),  
    FOREIGN KEY(codcid) REFERENCES cidade (codigo)  
)
```

Definir o atributo da FK

Depois dizer que este atributo é FK, e qual tabela ele referencia

Tabelas com chave alternativa

```
CREATE TABLE medico
(
    codigo integer not null,
    nome varchar (40),
    email varchar(20),
    CRM integer NOT NULL,
    codcid integer,
    PRIMARY KEY(codigo),
    FOREIGN KEY(codcid) REFERENCES cidade (codigo),
    UNIQUE (CRM)
)
```



Terá as mesmas restrições da PRIMARY KEY; só que não poderá ser referenciada por tabelas

Tabelas com chave alternativa

```
CREATE TABLE medico
(
    codigo integer not null,
    nome varchar (40),
    email varchar(20),
    CRM integer NOT NULL UNIQUE,
    codcid integer,
    PRIMARY KEY(codigo),
    FOREIGN KEY(codcid) REFERENCES cidade (codigo)
)
```

Alguns SGBDs permitem esta sintaxe
Postgresql aceita as duas sintaxes

Tabelas com PK composta

- ▶ Exemplo: a seguinte definição:

```
consulta (data, hora, codPac#, codMed#)  
    codPac REFERENCIA paciente (codigo)  
    codMed REFERENCIA medico (codigo)
```



Tabelas com PK composta

- ▶ Exemplo: a seguinte definição:

```
consulta (data, hora, codPac#, codMed#)  
    codPac REFERENCIA paciente (codigo)  
    codMed REFERENCIA medico (codigo)
```

- ▶ Em SQL, fica:

```
CREATE TABLE consulta  
(  
    data DATE not null,  
    hora TIME not null,  
    codPac INTEGER not null,  
    codMed INTEGER not null,  
    PRIMARY KEY(data, hora, codPac),  
    FOREIGN KEY (codpac) REFERENCES paciente (codigo),  
    FOREIGN KEY (codmed) REFERENCES medico (codigo)  
)
```

Observe: os 3 são NOT NULL

Referência a PK composta

- ▶ Exemplo: a seguinte definição:

```
cons_medicame (data, hora, codPac#, codMedica#)
```

```
codMedica REFERENCIA medicamento (codigo)
```

```
(data, hora, codPac) REFERENCIA consulta (data, hora, codPac)
```



Referência a PK composta

- ▶ Exemplo: a seguinte definição:

```
cons_medicame (data#, hora#, codPac#, codMedica#)  
  codMedica REFERENCIA medicamento (codigo)  
  (data, hora, codPac) REFERENCIA consulta (data, hora, codPac)
```

- ▶ Em SQL, fica:

```
CREATE TABLE cons_medicame  
(  
  data DATE not null,  
  hora TIME not null,  
  codPac INTEGER not null,  
  codMedica INTEGER not null,  
  PRIMARY KEY (data, hora, codPac, codMedica)  
  FOREIGN KEY (data, hora, codPac) REFERENCES consulta (data, hora,  
codPac),  
  FOREIGN KEY (codmedica) REFERENCES medicamento (codigo)  
)
```



Definição de CONSTRAINT

- ▶ As restrições de PK e FK podem ser nomeadas
- ▶ **Nomes únicos** em todo o BD

- ▶ A tabela abaixo tem uma PK sem nome:

```
CREATE TABLE cidade
(
    codigo integer not null,
    nome varchar (40),
    UF char(2),
    PRIMARY KEY (codigo)
)
```

- ▶ Já esta definição de tabela tem uma PK com nome “pk_cid”:

```
CREATE TABLE cidade
(
    codigo integer not null,
    nome varchar (40),
    UF char(2),
    CONSTRAINT pk_cid PRIMARY KEY (codigo)
)
```



Definição de CONSTRAINT

- ▶ A tabela abaixo tem uma FK sem nome:

```
CREATE TABLE medico
(
    codigo integer not null,
    nome varchar (40),
    codcid integer,
    CONSTRAINT pk_medico PRIMARY KEY (codigo),
    FOREIGN KEY (codcid) REFERENCES cidade (codigo)
)
```

- ▶ Já esta definição de tabela tem uma FK com nome “moradia”:

```
CREATE TABLE medico
(
    codigo integer not null,
    nome varchar (40),
    codcid integer,
    CONSTRAINT pk_medico PRIMARY KEY (codigo),
    CONSTRAINT moradia FOREIGN KEY (codcid) REFERENCES cidade (codigo)
)
```

Definição de valores *default*

- ▶ Toda coluna pode ter um valor *default*

- ▶ Exemplo:

```
CREATE TABLE cidade
(
    codigo integer not null,
    nome varchar (40),
    UF char(2) DEFAULT 'SC',
    CONSTRAINT pk_cid PRIMARY KEY (codigo)
)
```



Checagem de valores – CHECK ()

- ▶ Usado para criar restrições na criação da tabela
- ▶ Exemplo:

```
CREATE TABLE medico
(
    codigo integer not null,
    nome varchar (40),
    email varchar(20),
    CRM integer,
    codcid integer,
    PRIMARY KEY(codigo),
    FOREIGN KEY(codcid) REFERENCES cidade (codigo),
    CONSTRAINT const1 CHECK (email <> 'bla@' AND codigo > 100)
)
```



Checagem de valores – CHECK ()

- ▶ Usado para criar restrições na criação da tabela
- ▶ Exemplo:

```
CREATE TABLE paciente
(
    codigo integer not null,
    nome varchar (40),
    email varchar(20),
    categoria char(1) CHECK (categoria in ('A', 'B', 'C')),
    codcid integer,
    PRIMARY KEY(codigo),
    FOREIGN KEY(codcid) REFERENCES cidade (codigo)
)
```



Manipulação das tabelas

- ▶ CREATE TABLE

- ▶ Cria tabelas no Banco de Dados

- ▶ DROP TABLE

- ▶ Apaga tabelas já existentes no Banco de Dados

- ▶ ALTER TABLE

- ▶ Altera tabelas já existentes no Banco de Dados



Deleção de tabelas

- ▶ Comando

- ▶ `DROP TABLE nomeTabela;`

- ▶ Apaga toda a tabela e seu conteúdo

- ▶ Não há como recuperar a tabela removida

```
DROP TABLE medico;
```

```
DROP TABLE paciente;
```

```
DROP TABLE cidade;
```



**Cidade é referenciada por
medico e paciente
DEVE ser a última a ser deletada**



Alteração de tabelas

- ▶ **Comando**

- ▶ `ALTER TABLE nomeTabela`

- ▶ **Alterações possíveis**

- ▶ Adicionar/apagar coluna
 - ▶ Adicionar/apagar definição de valores *default*
 - ▶ Adicionar/apagar definição de PK e FK

- ▶ **Demais alterações (mudança de nome de coluna, mudança de tipo) são dependentes do SGBD**



Alteração de tabelas - colunas

▶ Exemplos

Medico

<i>codigo</i>	<i>nome</i>	<i>email</i>	<i>CRM</i>	<i>codCid</i>

▶ Adicionando coluna

```
ALTER TABLE medico  
ADD especializacao VARCHAR(40),  
ADD area VARCHAR(30);
```

Medico

<i>codigo</i>	<i>nome</i>	<i>email</i>	<i>CRM</i>	<i>codCid</i>	<i>especializacao</i>	<i>area</i>

▶ Apagando coluna

```
ALTER TABLE medico  
DROP especializacao;
```

Medico

<i>codigo</i>	<i>nome</i>	<i>email</i>	<i>CRM</i>	<i>codCid</i>	<i>especializacao</i>	<i>area</i>



Alteração de tabelas - *default*

▶ Exemplos

▶ Nova definição de *default*

```
ALTER TABLE cidade  
ALTER UF SET DEFAULT 'RS';
```

▶ Apagando definição de *default*

```
ALTER TABLE cidade  
ALTER UF DROP DEFAULT;
```



Alteração de tabelas – PK e FK

▶ Exemplos

▶ Adicionando definição de PK

```
ALTER TABLE paciente  
ADD PRIMARY KEY (codigo);
```

ou

```
ALTER TABLE medico  
ADD CONSTRAINT pk_paciente PRIMARY KEY (codigo);
```

▶ Apagando definição de PK

```
ALTER TABLE paciente  
DROP pk_paciente;
```

**Só funciona se tiver sido criada
uma *CONSTRAINT* para PK**

Alteração de tabelas – PK e FK

▶ Exemplos

▶ Adicionando definição de FK

```
ALTER TABLE paciente  
ADD FOREIGN KEY (codCid) REFERENCES cidade (codigo);
```

ou

```
ALTER TABLE paciente  
ADD CONSTRAINT mora FOREIGN KEY (codCid) REFERENCES  
cidade (codigo);
```

▶ Apagando definição de FK

```
ALTER TABLE paciente  
DROP mora;
```

**Só funciona se tiver sido criada
uma *CONSTRAINT* para FK**

Ordem na criação das tabelas

- ▶ Tabelas referenciadas:
 - ▶ Devem ser criadas antes

Exemplo:

Errado:

```
paciente (codigo, nome, codcid#)
          codcid referencia cidade (codigo)
cidade (codigo, nome, uf)
```

Correto:

```
cidade (codigo, nome, uf)
paciente (codigo, nome, codcid#)
          codcid referencia cidade (codigo)
```



DICA na criação de tabelas

Evita referências a tabelas ainda não criadas

1. Cria todas as tabelas sem chaves estrangeiras
 - Escrever todos os comandos de
`CREATE TABLE nomeTabela`
`(`
`...`
`);`
2. Alterar todas as tabelas que possuem referencias a outras tabelas, adicionando chaves estrangeiras
 - Escrever todos os comandos de

```
ALTER TABLE nomeTabela  
ADD CONSTRAINT nome  
FOREIGN KEY (nomeColunaChaveEstrangeira)  
REFERENCES tabela (nomeColunaChavePrimaria);
```


SQL/DML: manipulação de dados

- ▶ INSERT INTO . . .
 - ▶ Comando para **inserção** de dados nas tabelas
- ▶ UPDATE . . .
 - ▶ Comando para **alteração** de dados nas tabelas
- ▶ DELETE FROM . . .
 - ▶ Comando para **remoção** de dados nas tabelas
- ▶ SELECT . . .
 - ▶ Comando para **consulta** aos dados nas tabelas



Inserção de Dados

■ INSERT INTO

✓ **Sem informar** os atributos

Na tabela: Cidade (codigo, nome, uf)

```
INSERT INTO cidade VALUES (1, 'Passo Fundo', 'RS');
```

```
INSERT INTO cidade VALUES (2, 'Joenville', 'SC');
```



Inserção de Dados

■ INSERT INTO

- ✓ **Sem informar** os atributos
- ✓ **Informando** os atributos

Na tabela: Cidade (codigo, nome, uf)

```
INSERT INTO cidade VALUES (1, 'Passo Fundo', 'RS');
```

```
INSERT INTO cidade VALUES (2, 'Joenville', 'SC');
```

```
INSERT INTO cidade (codigo, nome) VALUES (3, 'Marau');
```

```
INSERT INTO cidade (codigo, nome) VALUES (4, 'Carazinho');
```



Inserção de Dados

```
INSERT INTO cidade VALUES (1, 'Passo Fundo', 'RS');
```

```
INSERT INTO cidade VALUES (2, 'Joenville', 'SC');
```

```
INSERT INTO cidade (codigo, nome) VALUES (3, 'Marau');
```

```
INSERT INTO cidade (codigo, nome) VALUES (4, 'Carazinho');
```

ou

```
INSERT INTO cidade (codigo, nome) VALUES (3, 'Marau'),  
(4, 'Carazinho'),  
(5, 'Palhoça'),  
(6, 'Florianópolis');
```

Resultado:

	CODIGO	NOME	UF
▶	1	Passo Fundo	RS
	2	Joenville	SC
	3	Marau	<null>
	4	Carazinho	<null>

Inserção de Dados

- **Inserção de chave estrangeira**
 - valor já deve existir na PK da outra tabela

paciente (codigo, nome, email, fone, codCid#)
codCid REFERENCIA cidade (codigo)

```
INSERT INTO paciente VALUES (1, 'Juca', 'ju@abc', '9999.9999', 1);
```

```
INSERT INTO paciente VALUES (2, 'Linda', 'll@abc', '9898.9898', 2);
```

```
INSERT INTO paciente (codigo, nome) VALUES (3, 'Luis');
```

```
INSERT INTO paciente (codigo, nome) VALUES (4, 'Lana');
```



Inserção de Dados

- **Inserção de chave estrangeira**
 - valor já deve existir na PK da outra tabela

paciente (codigo, nome, email, fone, codCid#)
codCid REFERENCIA cidade (codigo)

```
INSERT INTO paciente VALUES (1, 'Juca', 'ju@abc', '9999.9999', 1);  
INSERT INTO paciente VALUES (2, 'Linda', 'll@abc', '9898.9898', 2);
```



Inserção de Dados

```
INSERT INTO paciente VALUES (1, 'Juca', 'juc@abc', '9999.9999', 1);
```

```
INSERT INTO paciente VALUES (2, 'Linda', 'll@abc', '9898.9898', 2);
```

```
INSERT INTO paciente (codigo, nome) VALUES (3, 'Luis');
```

```
INSERT INTO paciente (codigo, nome) VALUES (4, 'Lana');
```

Resultado:

	CODIGO	NOME	EMAIL	FONE	CODCID
▶	1	Juca	juc@abc	9999.9999	1
	2	Linda	ll@abc	9898.9898	2
	3	Luis	<null>	<null>	<null>
	4	Lana	<null>	<null>	<null>



Inserção de Dados – abreviação dos comandos

```
INSERT INTO paciente VALUES (1, 'Juca', 'juc@abc', '9999.9999', 1),  
(2, 'Linda', 'll@abc', '9898.9898', 2),  
(3, 'Luis', 'lui@abc.def.gh, 1),  
(4, 'Lana', 'lana@abc.def.com, 2);
```

Observem a vírgula ao final, e não ponto e vírgula (que vai apenas no último comando).



Alteração de dados

- Supondo a tabela, com estes dados:

	codigo integer	nome character var	curriculo xml	idade integer	email character var
1	1	Aninha		30	ana@123.456
2	3	Juli		30	jul@abc.def
3	4	Lia		30	li@abc
4	2	Luca		17	luc@email.com



Alteração de dados

- Supondo a tabela, com estes dados:

	codigo integer	nome character var	curriculo xml	idade integer	email character var
1	1	Aninha		30	ana@123.456
2	3	Juli		30	jul@abc.def
3	4	Lia		30	li@abc
4	2	Luca		17	luc@email.com

UPDATE pessoa

SET idade = 18, email = 'jul@email.com'

WHERE nome = 'Juli';

	codigo integer	nome character var	curriculo xml	idade integer	email character var
1	1	Aninha		30	ana@123.456
2	4	Lia		30	li@abc
3	2	Luca		17	luc@email.com
4	3	Juli		18	jul@email.com

Exclusão de dados

Supondo a tabela:

	codigo integer	nome character var	curriculo xml	idade integer	email character var
1	1	Aninha		30	ana@123.456
2	4	Lia		30	li@abc
3	2	Luca		17	luc@email.com
4	3	Juli		18	jul@email.com



Exclusão de dados

Supondo a tabela:

	codigo integer	nome character var	curriculo xml	idade integer	email character var
1	1	Aninha		30	ana@123.456
2	4	Lia		30	li@abc
3	2	Luca		17	luc@email.com
4	3	Juli		18	jul@email.com

```
DELETE FROM pessoa  
WHERE nome = 'Lia';
```

	codigo integer	nome character var	curriculo xml	idade integer	email character var
1	1	Aninha		30	ana@123.456
2	2	Luca		17	luc@email.com
3	3	Juli		18	jul@email.com



Exclusão de dados

Supondo a tabela:

	codigo integer	nome character var	curriculo xml	idade integer	email character var
1	1	Aninha		30	ana@123.456
2	4	Lia		30	li@abc
3	2	Luca		17	luc@email.com
4	3	Juli		18	jul@email.com

DELETE FROM pessoa

	codigo integer	nome character var	curriculo xml	idade integer	email character var
--	---------------------------	-------------------------------	--------------------------	--------------------------	--------------------------------



Comentários em SQL

► Comentário de uma linha --

```
-- CREATE TABLE cidade (...)
```

► Comentário de um bloco /* ... */

```
/*CREATE TABLE cidade  
(codigo integer not null,  
...)  
*/
```



Resumo dos comandos

DDL

DML

<i>criação</i>	CREATE	INSERT
<i>alteração</i>	ALTER	UPDATE
<i>remoção</i>	DROP	DELETE



Exercícios

